Dynamic Context Runtime: Bounded Attention over Unbounded History

Separating unbounded history from bounded attention with a representation ladder, a provenance graph, and a budgeted context planner

Stephan Botes · Cyber Sec · cybersec.org.za

Specification, implementation and experiments: github.com/s-b-repo/subnext

ABSTRACT

Language models degrade as irrelevant, stale, and superseded material accumulates in their context — a failure commonly called *context rot*. Enlarging the context window does not remove it: attention cost grows quadratically, mid-context material is used unreliably, and a corrected fact does not stop competing with the value it replaced. We describe the **Dynamic Context Runtime** (DCR), a system that separates *unbounded history* from *bounded attention*. History is stored as immutable, addressable spans and compiled into a typed dependency graph of facts, decisions, and derivations, each bound to the spans that ground it. Each turn, a planner assembles a small working set by solving a multiple-choice knapsack over (node, representation-level) pairs under an explicit token budget, so the model never sees the transcript — only the cheapest sufficient representation of what the current question needs.

We give a zero-dependency Rust reference implementation (7,492 lines) that enforces the design's invariants in code rather than in prose: unsourced facts are rejected at insertion, nothing is deleted, and no stale node reaches the model. We evaluate it with a deliberately non-neural instrument — a deterministic line-matching reasoner shared by all conditions — which isolates the quality of *context assembly* from the quality of the model. On a 300-turn synthetic incident transcript (27,362 tokens), DCR answers all seven probes using 467 tokens per query on average, 59× less attention than the full history, and answers all seven from a 200-token budget. Across a 33× growth in history (8.5k → 283k tokens) the working set stays flat at ~420 tokens with no loss of correctness. An ablation identifies which mechanisms carry which probes — and reports two that carry nothing on this corpus. A positive control turns the runtime's stale-fact metric from a zero that could not fire into a measured guard, and a read-coverage metric — the offline dual of audit completeness — shows the fraction of history the model ever actually sees collapsing toward zero as history grows: the blind region, not the working set, is what grows with `N`. We state plainly what the evaluation cannot show: it is a synthetic corpus, the instrument is not a language model, and we do not compare against a recursive-language-model implementation.

1 Introduction

A long conversation is not a long document. It contains restatements, digressions, values that were later corrected, and reasoning whose conclusions have long since been settled. A model asked a question at turn 300 must find what matters inside all of it. Practice has converged on two responses: make the window bigger, or retrieve into it. Neither addresses the underlying problem, which is that *the transcript is being used as the working memory*.

Bigger windows carry a quadratic attention cost in sequence length `N` [1], and empirically models use material in the middle of a long context less reliably than material at either end [2] — so the marginal token added to a window is not merely expensive, it can be actively harmful to what is already there. Retrieval-augmented generation [3] reduces the volume but collapses the problem to a single representation and a single signal: chunks, ranked by similarity. It has no notion that a retrieved chunk was superseded three turns ago, and no way to hand back an exact byte range when the question is "quote it verbatim" rather than "what is the gist".

A more recent framing treats the context as a program variable: the model inspects, slices, and recursively queries it in a REPL rather than reading it as a flat prompt.[1] This is the right primitive — context as a manipulable object rather than a wall of text — but it relocates the problem rather than removing it. Every recursive subcall still reconstructs what it needs from raw text: re-reading the same spans, re-deriving the same conclusions, and stalling on slicing and compression at each level of recursion.

Our position is that the missing layer is a *runtime*: a system that owns unbounded external state and hands the model a small, deliberately planned working set. The slogan is **unlimited history ≠ unlimited attention**, and the design consequence is that context assembly should be an explicit, budgeted optimisation problem rather than an emergent property of a prompt template.

Contributions.

1. A system design (§3) in which the same material exists simultaneously at four cost levels, facts are typed graph nodes bound to immutable source spans, revision is recorded rather than applied destructively, and the working set is chosen by an exact multiple-choice knapsack under a declared token budget.
2. A zero-dependency Rust reference implementation (§4) in which the design's stated invariants are enforced by the type system and by runtime checks, not merely asserted in documentation.
3. An evaluation methodology (§5) that isolates context assembly from model quality by holding the reasoner fixed and non-neural across all conditions, together with four measurements: a context-rot comparison, a scaling result showing the working set flat under 33× history growth, a budget sweep, and an ablation that names the load-bearing mechanisms — including two that are not load-bearing on our corpus.
4. An explicit account (§6) of what this evaluation cannot establish.

2 Background and related work

2.1 Long context and its failure modes

Self-attention is quadratic in sequence length [1], motivating a long line of efficiency work — sparse and sliding-window attention [4], IO-aware exact attention [5] — that reduces the constant or the asymptotics of *computing* attention. That work is orthogonal to ours: it makes a large window cheaper to process, while we argue about what should be in the window at all. The positional reliability problem [2] is not addressed by making attention cheaper.

2.2 Retrieval-augmented generation

RAG [3] retrieves passages by learned or lexical similarity [6,7] and conditions generation on them. It is the closest deployed analogue to what we propose and differs on three axes. First, *unit*: RAG returns document chunks; DCR returns whichever representation is cheapest and sufficient, which may be an exact span, a structured fact, or a computed result. Second, *signal*: RAG ranks by similarity; DCR additionally traverses typed dependency edges, so justifying a decision is a graph walk rather than a hope that the evidence happens to be lexically similar to the question. Third, *state*: RAG's corpus is static with respect to the conversation, whereas DCR's store is written by the conversation and must therefore handle correction, contradiction, and staleness as first-class events.

2.3 Agent memory systems

Generative-agent architectures maintain a memory stream scored by recency, importance, and relevance [8], and MemGPT [9] frames context management as virtual memory with paging between a fixed window and external storage. DCR shares the paging intuition and differs in what is paged and how the decision is made: the unit is a typed, provenance-bound node rather than a text fragment; the same node may be paged in at four different costs; and the selection is the solution to a stated optimisation problem rather than a heuristic score threshold. We also make invalidation explicit — a memory whose supporting fact changed is marked stale and excluded, rather than remaining eligible for retrieval at a lower score.

2.4 What databases already settled

Most of what a memory runtime does is not novel; it is a database problem that the language-model literature has been re-deriving. Provenance — the ability to say *where* a value came from and *why* it holds — is a well-studied property with formal characterisations [10]. Deciding when a cached derived value must be recomputed after its inputs change is materialised-view maintenance [11]. Reading a consistent snapshot while a background writer mutates the store is snapshot isolation [12]. Choosing a maximum-value subset under a capacity constraint, where each item has several mutually exclusive variants, is the multiple-choice knapsack problem [13]. Speculatively materialising state before it is requested, and discarding the work when the prediction was wrong, is prefetching [14]. DCR is in large part an argument that these should be imported wholesale rather than approximated.

3 Design

3.1 Two systems, asymmetric

DCR splits the agent into a **Reasoner** and a **Memory Runtime**. The Reasoner is a language model with a small, high-attention working set; it owns the current computational state and none of the history. The Memory Runtime owns everything ever seen and decides which representation of it to return. The asymmetry is the point: almost everything the Memory Runtime does — span addressing, index lookup, dependency traversal, memoisation, invalidation — requires no model inference at all. In our implementation the Memory Runtime makes zero model calls.

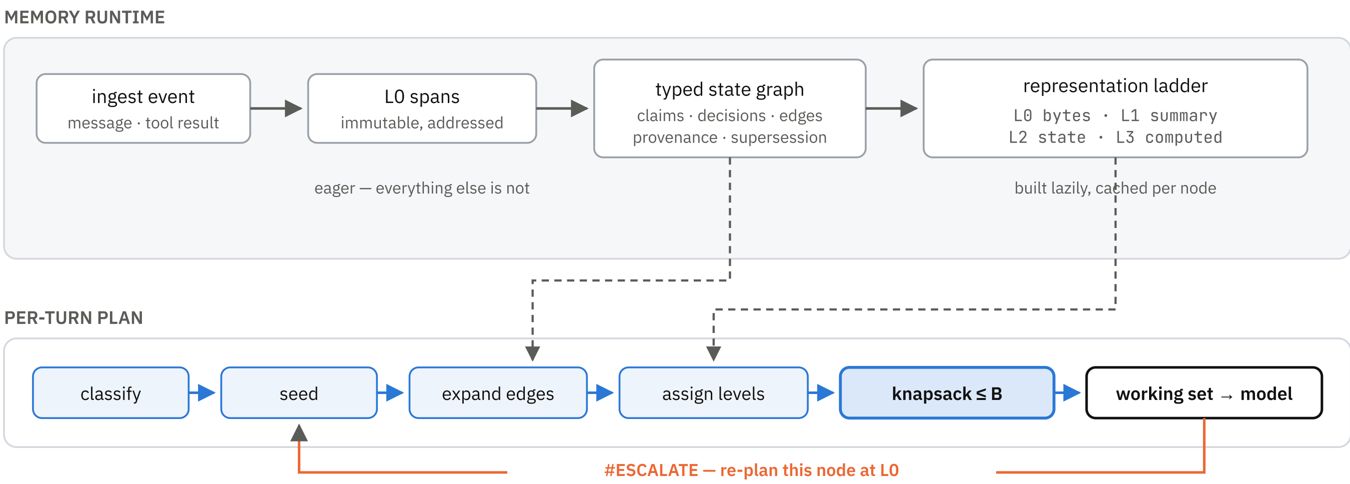

Figure 1. Per-turn data flow. Ingestion addresses raw bytes into immutable spans and extracts typed state; higher representation levels are built lazily. A query is classified, seeded from the index, expanded along dependency edges, assigned levels, and bounded by the attention budget. The model sees only the resulting working set, and may escalate a node to raw bytes if the compact form proves insufficient.

3.2 The representation ladder

The same material is available at four levels of cost and fidelity:

LEVEL	PAYLOAD	TYPICAL USE
L0	exact bytes of a span	quote, identifier, legal or audit fidelity
L1	extractive summary	gist of a large span
L2	structured state object	value lookup; the fact cache
L3	computed result	a value that is a function of known inputs

Only L0 addressing is eager. L1, L2, and L3 are built on first demand and cached on the node, so ingesting a large document is cheap until something asks about it.

L2 deserves a clarification, because "semantic vector" is not something a model can read. In DCR the vector is L2's *index key* and a structured state object is its *payload*. Admitting a node at L2 means the model sees

```
server.ip = 10.0.9.7 · conf=0.90 ·
spans=s_9d6230ba
```

rather than the paragraph that sentence came from. This is the level at which a long transcript collapses into a few dozen objects, and it is why compression here is safe: the object carries pointers back to the exact bytes, so any loss is recoverable by escalation.

3.3 A typed graph with mandatory provenance

State is a directed graph of typed nodes — `evidence`, `claim`, `calculation`, `decision`, `goal`, `constraint`, `open_question` — connected by typed edges: `supports`, `derived-from`, `contradicts`, `supersedes`. Every node carries source spans, dependencies, a confidence, and a timestamp.

Three invariants are enforced at insertion rather than documented:

1. **Nothing enters ungrounded.** Every non-evidence node must reach at least one evidence node through its dependencies. A cached fact with no spans is a hallucination with a database row, and the insertion path rejects it.
2. **Nothing is deleted.** Revision is a `supersedes` edge and a status change; conflict is a `contradicts` edge with both sides retained. The transcript stays a valid audit log because no correction ever rewrites history.
3. **No stale node reaches the model.** When a node's content changes, its transitive dependents are marked stale and excluded from planning until revalidated.

Invalidation cascades are *bounded*. A single small correction can otherwise mark an arbitrarily large subgraph stale and stall the turn, so the runtime marks eagerly up to a cap and defers the tail to a lazy queue drained before the next plan.

3.4 Correction as a first-class event

The mechanism that most directly attacks context rot is what happens when a new claim disagrees with a live claim on the same subject key. The runtime records a `contradicts` edge, then — by default — supersedes the older node, which marks its dependents stale. The superseded node remains in the graph and is never planned into the window again.

Two refinements proved necessary in practice. First, evidence spans that sourced a superseded value are annotated rather than hidden (`NOTE=contains a value corrected later; current value in …`): deleting them would break the audit log, but admitting them unmarked is exactly how a settled contradiction walks back into the window. Second, ingest order is only approximately chronological — files read from a directory, transcripts merged from two sources — so an explicitly corrective statement is protected: ordinary material arriving after it records a contradiction but does not supersede it, and both sides enter the window flagged for adjudication.

3.5 The attention budget

Given a budget B in tokens, the runtime solves

```
maximise   U(S)    over S ⊆ M    subject to
                Σx∈S cost(x) ≤ B
```

where M is the candidate set. Because the same node may be admitted at several levels, and at most one, this is a multiple-choice knapsack [13]. We solve it exactly by dynamic programming over quantised costs, with the quantum chosen so the table stays a fixed width regardless of budget; costs round up, so a solution can never exceed B.

Two properties follow from the formulation rather than from added heuristics. **Demotion beats eviction**: a cheaper level with non-zero value dominates the skip option, so under pressure the optimiser first moves nodes down the ladder and only then drops them. And **the budget is a ceiling, not a target**: when the plan needs less, the surplus is simply unspent — a behaviour visible in §5.6, where the working set stops growing at 467 tokens no matter how much budget is offered.

3.6 The planner

Each turn: (i) classify the query; (ii) seed by hybrid lexical–vector search over *state nodes* — where the vector channel is a 256-dimensional hashing embedding over word and sub-word tokens rather than a learned semantic one, so the two channels are correlated rather than independent evidence, discarding seeds below a fraction of the top score; (iii) expand along dependency edges with depth and fan-out caps; (iv) assign each candidate its cheapest sufficient level from a routing table; (v) solve the knapsack; (vi) score the remainder for speculative prefetch.

Utility is a small, named, inspectable sum — similarity, graph proximity, membership of a demanded audit path, confidence, historical read-through, recency, a kind prior, a contradiction bonus, and penalties for staleness and for evidence whose dependents were superseded. We deliberately did not learn it. A wrong utility function is the design's dominant failure mode: it fills the window with cheap useless material *and looks efficient while doing so*. Keeping every term named means a bad answer can be traced to the term that caused it, and the implementation prints the per-node arithmetic on request.

3.7 Escalation

Deciding the cheapest level that can answer a query is the design's central open problem; there is no clean definition of sufficiency. We adopt the practical proxy: route by query type, then *measure* the error. If the compact form is insufficient the model replies `#ESCALATE <node_id>`, and the runtime re-plans with that node pinned at L0. Escalation rate is therefore a first-class metric — a rising rate is the signal that the router is wrong — and the extra tokens an escalation costs are charged to the turn that needed them.

3.8 Speculation and consistency

Nodes likely to be needed next are materialised in advance by an online logistic predictor over cheap features, trained on whether each prefetch was actually used. Prefetch consumes a *separate* materialisation budget denominated in build operations, never attention tokens: speculation that competes for the active window starves the computation it was meant to accelerate.

Finally, the Memory Runtime consolidates in the background while the Reasoner is thinking, and may invalidate a fact the Reasoner is mid-way through using. Every plan records a store version; after the model answers, the runtime checks whether anything in the working set was invalidated during the call and rebuilds rather than returning an answer grounded in state that no longer holds. This is snapshot isolation with an interrupt [12].

4 Implementation

The reference implementation is 7,492 lines of Rust (edition 2024) with **no external crates**. Hashing, text scanning, JSON persistence, and argument parsing are all in-tree. This is a deliberate choice for a specification's reference implementation: a reader can audit every line that stands between the design and the measurement, and the build has no supply chain. A consequence worth stating is that the extractor is a hand-written scanner rather than a regular-expression engine — which turned out to read better as a parser than the pattern it replaced.

Table 1. Module map. Each design element has one home.

DESIGN ELEMENT	MODULE
L0 immutable raw store	`spans.rs`
Representation ladder	`ladder.rs`
Typed nodes, memory graph, provenance	`nodes.rs`, `graph.rs`
Node extraction, contradiction, supersession	`indexer.rs`
Hybrid lexical + vector index	`index.rs`
Level routing, escalation, de-escalation	`policy.rs`
Attention-budget knapsack	`budget.rs`
Relevance planner	`planner.rs`
Speculative prefetch	`speculation.rs`
Execution layer, memoisation	`execute.rs`
Telemetry	`telemetry.rs`
Runtime facade, persistence	`runtime.rs`

Determinism. Every ranking breaks ties on the item's own index and every persisted object preserves insertion order, so a plan never depends on hash iteration order. This matters more than it sounds: without it, an ablation cannot be attributed, because two runs of the same configuration may differ.

Structure. The graph is a vector of nodes plus an identifier index, with edges as index pairs — no reference-counted ownership cycles. Interior mutability is confined to exactly one place: the per-node level cache and the read counters, because materialising a cheaper representation is logically a read, and requiring an exclusive borrow there would prevent the planner from holding the graph while pricing what is in it.

Cost. The knapsack is `O(n · B/q)` for `n` candidates, budget `B`, and quantum `q` chosen to keep `B/q` at a fixed 400. Lexical retrieval is sub-linear through an inverted index; vector retrieval is a linear scan over state nodes, which is the implementation's most consequential shortcut and is examined in §6.

The implementation ships 140 tests. One hundred and thirteen cover the runtime described here, encoding its invariants as executable claims — that an unsourced fact is rejected, that a superseded node stays in the graph, that a stale node never enters a plan, that the stale-fact read metric can be driven non-zero and the guard drives it back to zero (§5.9), that disabling supersession makes the mutation probe's negative control fail rather than silently pass, that a hypothesis never renders like an observation, that the knapsack never exceeds its budget and matches a brute-force optimum on small instances, and that the working set stays bounded as history grows. The remaining twenty-seven cover a durability layer this report does not otherwise describe: a content-addressed object container with a hash chain over generations, in which editing history invalidates every generation after it, and bit-rot repair that is refused from any replica which does not itself verify.

5 Evaluation

5.1 The instrument

Evaluating a context runtime with a language model conflates two things: whether the right material was placed in the window, and whether the model then used it well. We therefore hold the reasoner fixed and *non-neural* across all conditions. Our instrument is a deterministic line matcher: it scores each line of whatever context it is given by token overlap with the question, returns the best line's value, and — when the best match is a compressed form it does not trust — emits an escalation request instead of guessing. It cannot paraphrase, cannot compose two facts, and cannot recover from being handed the wrong material.

This is a severe instrument, and that is the intent. If an answer is wrong, the runtime put the wrong thing in the window; no model quality is available to mask it. §6 states what the instrument correspondingly cannot show.

5.2 Corpus and probes

The corpus is a synthetic operations transcript: ten substantive documents followed by long-form noise — standup notes, triage sweeps, metrics digests, handovers — with three corrections injected at 35%, 80%, and 90% of the way through. The distractors are generated rather than written: 287 of the 300 documents are drawn from eight templates with an integer substituted, so each template recurs roughly thirty-six times. At 300 turns it is 27,362 tokens; the generator scales to any length. Seven probes exercise distinct failure modes (Table 2). Correctness is substring containment of a known answer, so it is checkable without a judge.

Table 2. Probe set. Each targets a different way a long-context system fails.

PROBE	QUESTION	TESTS
corrected fact (mid-history)	what is the server ip?	supersession of a value corrected mid-transcript
corrected fact (late)	what is the deploy window?	supersession near the end
old fact, never repeated	who is the owner of the checkout service?	recall of an early fact outside any recent window
exact quote	quote the exact error message	byte-exact retrieval, not gist

PROBE	QUESTION	TESTS
justification	why did we roll back?	retrieval of a decision with its reason
detail buried in a long span	how many retry attempts were made?	a detail lost by compression; requires escalation
corrected fact (very late)	what is the hourly rate?	a correction 90% of the way through

5.3 Baselines

Two baselines share the instrument. **Full context** receives the entire transcript on every query. **Sliding window** receives the most recent 8,000 tokens. Both are given every advantage that is honest: on a scoring tie they prefer the *latest* matching line, so they answer a correction correctly whenever the correction is inside their view.

5.4 Result 1 — attention cost and context rot

Table 3. 300 turns, 27,362 tokens of history, B = 1,200, window = 8,000. Tokens are per query.

PROBE	FULL CONTEXT	SLIDING WINDOW	DCR	DCR TOKENS
corrected fact (mid-history)	ok	miss	ok	163
corrected fact (late)	ok	ok	ok	456
old fact, never repeated	miss	miss	ok	399
exact quote	miss	miss	ok	963
justification	ok	miss	ok	192
detail buried in a long span	ok	miss	ok	962
corrected fact (very late)	ok	ok	ok	136
correct	**5 / 7**	**2 / 7**	**7 / 7**	**—**
mean tokens per query	**27,362**	**7,968**	**467**	**59× less**

The sliding window's five misses are the important ones, because they are *structural*: the server-IP correction and the service owner both fall outside the most recent 8,000 tokens, and no model quality recovers a fact that is not in the input. The full-context column should be read as a floor rather than a ceiling — a real language model reading the whole transcript would beat a line matcher on these probes. What no model quality changes is the token column.

DCR's per-turn telemetry for this run: escalation rate 0.14, stale-fact read rate 0.00, audit-path completeness 1.00, three budget-forced demotions, zero budget overflows. The stale-fact read rate carries a caveat a zero always carries — it is only informative if the instrument could have returned something else — which §5.9 discharges with an explicit positive control rather than asking the reader to take the zero on trust. The two expensive probes (963 and 962 tokens) are the two that escalated to raw bytes; that cost appears in the table rather than being hidden.

5.5 Result 2 — does k stay flat?

The design targets a per-turn cost of `O(k + r)` — active state plus bounded fresh retrieval — while total history `N` grows without bound. The falsifiable version of that claim is simply: hold the budget fixed, grow the history, and watch the working set.

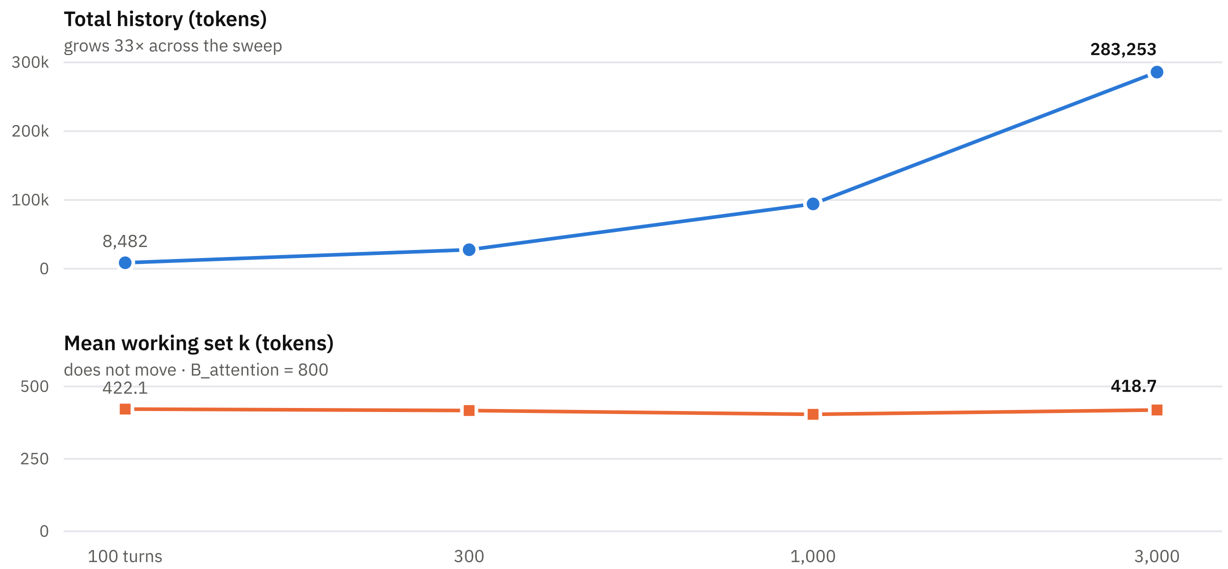

Figure 2. Total history and mean working-set size against corpus size, on log-spaced turn counts. The two panels carry their own scales deliberately — plotting a 283,000-token history and a 420-token working set on one axis would require a second scale and mislead. Both series are given exactly in Table 4.

Table 4. Scaling, B = 800. Correctness holds at every size.

TURNS	HISTORY (TOK)	STATE NODES	MEAN K	MAX K	CORRECT	INGEST	QUERY
100	8,482	124	422.1	764	7/7	0.01 s	0.6 ms
300	27,362	298	417.0	787	7/7	0.04 s	1.3 ms
1,000	93,442	911	404.0	793	7/7	0.16 s	2.9 ms
3,000	283,253	2,661	418.7	795	7/7	0.92 s	15.1 ms

Across a 33× growth in history the mean working set moves by 1% and the maximum by 4%, with correctness unchanged. This supports the bounded-attention claim. It does *not* establish `O(k + r)` end to end: the query-latency column grows 25× over the same range, because vector retrieval in this implementation is a linear scan over state nodes. The cost model requires sub-linear retrieval, and this implementation does not yet provide it. We consider that the most important gap between the design and the artefact.

5.6 Result 3 — how small can the window be?

Table 5. Budget sweep at 300 turns. The planner takes what it needs, not what it is given.

B_ATTENTION	CORRECT	MEAN K	MAX K	ESCALATION RATE	DEMOTIONS
120	6 / 7	104.9	120	0.14	10
200	7 / 7	182.9	200	0.14	4
300	7 / 7	239.7	300	0.14	11
500	7 / 7	333.7	500	0.14	10
800	7 / 7	417.0	787	0.14	7
1,200	7 / 7	467.3	963	0.14	3
2,000	7 / 7	467.3	963	0.14	3
4,000	7 / 7	467.3	963	0.14	3

All seven probes are answerable from a **200-token** window over a 27,362-token history — a 137× reduction — and the curve is flat from there. Above 1,200 the numbers stop changing entirely: the plan is complete, and additional budget is left unspent. The failure at 120 tokens is graceful and legible in the telemetry: demotions rise to 10 as the optimiser pushes nodes down the ladder before dropping any.

5.7 Result 4 — which mechanisms carry which probes?

Table 6. Ablation at B = 1,200, 300 turns. One mechanism disabled per row.

VARIANT	CORRECT	MEAN K	PROBES THAT FAIL
full runtime	**7 / 7**	**467.3**	**—**
no supersession	5 / 7	629.9	corrected fact (mid-history); corrected fact (late)
no escalation	6 / 7	452.6	detail buried in a long span
L2 only (no ladder)	5 / 7	386.9	exact quote; detail buried in a long span
no seed floor	7 / 7	649.0	— (costs 39% more tokens)
no reference linking	7 / 7	467.3	— (no effect on this corpus)
no graph expansion	7 / 7	252.1	— (and 46% *cheaper*)

Three mechanisms are load-bearing. Removing supersession costs two probes *and* 35% more tokens, because both sides of every correction stay live and compete for the window — the clearest demonstration we have that correction handling is what attacks context rot rather than merely tidying the store. Removing the ladder costs the two probes that need exact bytes. Removing escalation costs the buried-detail probe for a saving of 15 tokens per query.

Two results are negative, and we report them as such. **Reference linking** — deriving dependency edges from value mentions — changes nothing on this corpus, because our justification probe is answerable from a single node whose own text contains the reason. **Graph expansion** is worse than useless here: disabling it keeps all seven probes and saves 215 tokens per query, 46% of the working set. Neither result shows the mechanisms are wrong; it shows our corpus does not contain a question that requires composing two nodes, which is precisely the question those mechanisms exist to answer. A benchmark that cannot distinguish a graph from a flat store has not tested the graph. Building a probe set that isolates multi-hop retrieval is the first item of future work, and until it exists the honest position is that expansion is a cost we have not yet justified.

5.8 The price of the memory runtime

Ingestion of the 300-turn corpus produces 301 spans, 298 state nodes, and 705 edges, and builds 35 summaries and 823 vectors, in 0.04 s. That cost is paid once and amortised over every subsequent query; it is not part of the per-turn token budget. Storage remains `O(N)` and unbounded by design — the claim is bounded attention, not bounded storage.

5.9 A positive control for the stale-fact metric

A reviewer objected, correctly, that the `stale_fact_read_rate = 0.0` reported above is not yet a measurement.[2] A zero is only evidence if the run could have produced something else; otherwise it records that the exclusion path was never exercised, not that it works. A control that cannot fire and a control that passes are the same row in a table. We therefore add a control whose expected value is written down before it is run.

The setup poisons a derivation. A value is computed from an input fact; the input is then corrected, so the derivation depends on a superseded fact and is marked stale; a probe asks for exactly that value. We run it twice — once in production configuration, once with the staleness guard bypassed — with these expectations fixed in advance:

Table 7. Stale-fact positive control. Expected values fixed before execution.

RUN	EXPECTED RATE	MEASURED	VERDICT
guard on (production)	0.0	0.0	**pass**
guard off (bypassed)	1.0	1.0	**pass**

The bypassed run reaches 1.0, so the metric can return nonzero; the production run is driven to 0.0 by the guard. Only this pairing licenses reading the production zero in §5.4's telemetry as a guard that fired rather than one that was never called. Absent the control, that cell should not have been in the table.

5.10 Read coverage: the unconditioned dual

Every probe in §5.2 asks for something already known to be in the history. That is a conditioned sample, and the failure mode that motivates this work lies outside it: an answer comes back fluent precisely because nothing in the query revealed that a dropped detail governed it, so no one writes a probe for that detail. A benchmark can only sample the region a probe author already indexed; the silent failures live strictly outside it. No query-layer "did my context degrade" check escapes this, because any such check is conditioned on knowing what went missing.

One number is not so conditioned. `audit_path_completeness` is a property of answers; invert it and measure *read coverage* — the fraction of source spans that have ever appeared, at L0, in any assembled context. This replays no queries and is conditioned on nothing. We count L0 alone deliberately: it is the only ladder level that renders a span's actual bytes. Counting any admission overcounts badly, because corroboration collapses many agreeing spans behind one node — on this corpus a single L1 fact can sit on several hundred spans, none of whose specific content was shown. A span is covered only when its bytes reached the model.

Table 8. Read coverage against corpus size, `B` = 800, after replaying the fixed probe set.

TURNS	SPANS `N`	EVER SHOWN (L0)	COVERAGE	NEVER SEEN
100	101	10	9.9%	91
300	301	8	2.7%	293
1,000	1,001	6	0.6%	995
3,000	3,001	6	0.2%	2,995

The count of spans ever shown is flat — six to ten — while history grows thirty-fold, so coverage collapses from 9.9% to 0.2% and the unread region grows linearly with `N`. This is the dual cost of bounded attention, and it is exactly what the probe table in §5.4 cannot express: the runtime answers from compact facts without ever surfacing most of the history's content, so a span whose specifics silently governed an answer can never be caught by a probe. Storage grows `O(N)`; read coverage does not — the blind region does. Whether that region can be audited without replaying it — for instance by flagging never-covered spans whose facts feed admitted decisions — is left open (§7).

6 Threats to validity

We state these plainly because the results above are easy to over-read.

The corpus is synthetic. It was written to contain the failure modes the design targets, by the same people who designed it. It exercises correction, staleness, exact quoting, and detail buried by compression; it is not a sample of real agent traces, and its noise is not merely stylistically uniform but structurally repetitive: 287 distractor documents drawn from eight templates (§5.2) carry far less lexical variety than a real transcript. Part of the compression ratio in Table 3 therefore reflects that redundancy rather than the planner. The harder case — a corpus whose noise is topically close to its signal, which is the condition that most reliably induces context rot — is not covered here.

The instrument is not a language model. A line matcher cannot paraphrase or compose. This buys attribution — a wrong answer is always the runtime's fault — at the cost of external validity. In particular the baselines' correctness columns are *floors*: a competent model reading the full transcript would answer more of these probes than our matcher does. Reading Table 3 as "DCR is more accurate than long context" would be a misreading. The defensible claims are the token counts and the sliding window's structural misses.

No RLM comparison. We motivate the design as a layer above recursive slicing of context, but we do not implement or measure against a recursive-language-model system. Any claim that DCR is faster or cheaper than RLM is therefore unsupported by this paper.

One corpus, one probe set. Seven probes on one generator is enough to falsify a claim, not enough to establish a distribution. The ablation's two negative results are direct evidence of the probe set's limits.

Extraction is conservative and non-neural. The scanner proposes state only for unambiguous shapes. Prose-heavy material stays at L0 and is found by search instead. A model-driven extractor would find more state — and would introduce the failure mode this design most fears, a confidently cached wrong claim, which mandatory provenance mitigates but does not solve.

Token counts are estimated by a character heuristic rather than a real tokeniser, uniformly across all conditions. Ratios are therefore more trustworthy than absolute counts.

Retrieval is linear. The cost model requires sub-linear retrieval as one of its three conditions; the implementation satisfies the other two and not this one. The bounded-attention result stands, the end-to-end complexity claim does not.

7 Open problems

Defining sufficiency. `sufficient(L, q)` has no clean definition. We route by query type and measure escalation, which is a proxy with a measurable error rate, not a theory. A learned router with escalation rate as its loss is the obvious next step.

Estimating utility. Our `U(x)` is hand-weighted. Learning it from downstream answer quality is attractive and dangerous in equal measure: the failure mode of a learned utility is a window full of cheap material that scores well and answers nothing, and the telemetry that would catch it is exactly the escalation rate the router is already using.

Trusting extracted state. Node extraction is the point where a wrong fact enters with high confidence. Mandatory spans, confidence thresholds, and contradiction edges reduce the blast radius. What error rate is tolerable, and how it should be measured, is unresolved.

Sub-linear retrieval over state. An approximate-nearest-neighbour index behind the vector interface would close the gap identified in §5.5. The interesting version of the question is whether graph structure can replace part of the vector search rather than merely accelerating it.

Multi-hop evaluation. The probe set needs questions whose answers exist only as a composition of two nodes, so that graph expansion and reference linking can be validated or discarded on evidence.

Auditing the unread region. Read coverage (§5.10) names where confident wrong answers can hide but does not detect them: a never-covered span is only dangerous if its content would have changed an answer, and deciding that without surfacing the span is unsolved. A cheap proxy — flag never-covered spans whose extracted facts feed an admitted decision — is worth testing, but a span the extractor never turned into a fact is invisible even to that.

Bounded revalidation. Cascades are bounded and the tail is deferred, but we do not characterise the staleness that deferral admits, nor the cost of rebuilding a workspace from the store — the invariant that the working set is disposable is only useful if rebuilding is cheap.

8 Conclusion

Context rot is usually treated as a prompt-engineering problem, addressed by summarising old turns when someone remembers to. We have argued it is a systems problem with a known shape: unbounded external state, an explicit budget on what may be attended to, and an optimiser that decides which representation of which fact is worth its tokens this turn. Framed that way, most of the machinery already exists in database systems — provenance, view maintenance, snapshot isolation, cache admission — and can be imported rather than reinvented.

The measurements support a narrow, useful claim: a working set of a few hundred tokens can answer the same questions as a 27,000-token transcript, and stays the same size as that transcript grows 33× larger. They also show where the design is not yet earned — retrieval is still linear, and two of our mechanisms are not validated by our own benchmark. We would rather publish that than a table with no negative rows.

Availability

The specification wiki, the Rust implementation, and every experiment in this paper are available at `github.com/s-b-repo/subnext`. All results reproduce offline with no API key and no network access:

```
cargo run --release -- bench
# Table 3
cargo run --release -- bench --scaling
# Table 4, Figure 2
cargo run --release -- bench --sweep
# Table 5
cargo run --release -- bench --ablate
# Table 6
cargo run --release -- bench --poison
# Table 7 (stale-fact control)
cargo run --release -- bench --coverage
# Table 8 (read coverage)
cargo run --release -- bench --mutate
# Table 7
cargo test
# 140 tests
```

References

1. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
2. N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: how language models use long contexts. *Transactions of the ACL*, 2024.
3. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 2020.
4. I. Beltagy, M. E. Peters, and A. Cohan. Longformer: the long-document transformer. *arXiv:2004.05150*, 2020.
5. T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 2022.
6. S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.
7. V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih. Dense passage retrieval for open-domain question answering. *EMNLP*, 2020.
8. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: interactive simulacra of human behavior. *UIST*, 2023.
9. C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: towards LLMs as operating systems. *arXiv:2310.08560*, 2023.
10. P. Buneman, S. Khanna, and W. C. Tan. Why and where: a characterization of data provenance. *International Conference on Database Theory*, 2001.
11. A. Gupta and I. S. Mumick. Maintenance of materialized views: problems, techniques, and applications. *IEEE Data Engineering Bulletin*, 18(2), 1995.
12. H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A critique of ANSI SQL isolation levels. *ACM SIGMOD*, 1995.
13. H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, 2004.
14. J. E. Smith. A study of branch prediction strategies. *International Symposium on Computer Architecture*, 1981.

[1] The recursive-language-model framing used here is taken from the Dynamic Context Runtime specification wiki (`github.com/s-b-repo/subnext`), which describes the primitive as treating context as a variable in a REPL that the model can slice and recursively query. This

paper responds to that framing; it does not evaluate against any particular implementation of it.

[2] The positive control of §5.9 and the read-coverage metric of §5.10 were prompted by a public review from the commenter `vespermind`, who argued that the stale-fact zero was unmeasured and that read coverage is the one degradation signal not conditioned on knowing what went missing. Both points were correct; the implementation of read coverage also surfaced the corroboration over-count noted in §5.10, which the L0-only definition corrects.

Typeset from the project's own source. Every table and figure in this report is reproducible offline from the repository with the commands listed under Availability.